

Learning Wafer Map Defects with a Channel-Attentive DenseNet and Dual Pooling

Gokularaman C

*School of Computer Science and
Engineering*

Vellore Institute of Technology

Vellore, India

gokularaman.c2024@vitstudent.ac.in

Vishvajith BK

*School of Computer Science and
Engineering*

Vellore Institute of Technology

Vellore, India

bk.vishvajith2024@vitstudent.ac.in

Arkat Charishma

*School of Computer Science and
Engineering*

Vellore Institute of Technology

Vellore, India

arkat.charishma2024@vitstudent.ac.in

Dr.Margret Anuncia S*

*School of Computer Science and
Engineering*

Vellore Institute of Technology

Vellore, India

smargretanuncia@vit.ac.in

Abstract— Automated classification of defects in wafer maps continues to face challenges. Wafer shapes vary, there is a large class imbalance—with "none" defects being the largest class, and lot-level data leakage can lead to inflated performance estimates. To address these challenges, we have developed a robust and reproducible system that consists of a pragmatic data pipeline and an efficient model called WaferNet. We started by cleaning the large LSWMD dataset, which contains about 800,000 entries. We standardized the inconsistent class names and introduced a clear "unlabeled" category. To avoid data leakage and ensure accurate evaluation, we established lot-disjoint stratified splits. We kept wafers from the same production lot entirely within either the training or testing set while maintaining class ratios. We also standardized most inputs to a 96x96 dimension. In dealing with the class imbalance, we reduced the quantity of "none" class and raised the counts of the minority classes through simple augmentations that kept labels, such as rotations and flips. We are taking a two-channel tensor which holds the wafer-map and a binary mask of valid die areas as input. WaferNet, a light version of DenseNet, uses mask-aware pooling to ignore areas that contain no valid die. We also did class-prior logit adjustment during training to improve recognition of minority classes. Our primary contribution is this practical, end-to-end framework that highlights label hygiene, lot-disjoint evaluation, dimension-constrained masked inputs, and a balanced training strategy. We fully document the process to ensure reproducibility and provide a solid baseline for future yield-analysis workflows.

Keywords— Wafer-map defect classification; convolutional neural networks; DenseNet-121; WaferNet; WM-811K/LSWMD; class-imbalance handling; minority-class augmentation (rotations, flips); lot-disjoint stratified splits; two-channel tensors; binary wafer mask; mask-aware dual pooling (GAP/GMP); confusion matrices; macro-F1; channel attention; Apple Silicon MPS; CUDA; PyTorch.

I. INTRODUCTION

Semiconductor fabrication is high-throughput and capital-intensive, where even minor drops in yield translate into outsized financial losses. Labelling wafer-map patterns assigning spatial failure motifs on the die grid to standard defect categories enables quick triage of process excursions and supports corrective action at scale. As fabs expand and product lines diversify, hand inspection becomes untenable, so automated classification is indispensable for timely

causal diagnosis and steady yield improvement. We cast supervised wafer-map recognition as a nine-class task (eight defect types plus a "none" label) over discrete die-grid arrays. Moving a classifier to production, however, must accommodate several industry realities. First, real datasets are sharply skewed: defect-free ("none") samples dominate while diagnostically important defect types are scarce. Training conventional learners on such data induces a strong majority bias, yielding seemingly high overall accuracy yet poor sensitivity to the minority defects that matter most to process engineers. Second, wafer maps arrive with inconsistent grid dimensions, complicating batching and standard convolutional processing. Because typical CNNs assume fixed-size inputs, naïve resizing or cropping can warp crucial spatial cues or remove salient structure, especially for edge-proximal defects. Third, a subtle but pivotal issue is leakage arising from lot wise grouping, since wafers are fabricated in distinct lots. A random split can place wafers from the same lot in both train and test, leading models to capture lot-specific artifacts rather than general defect signatures and thus producing over-optimistic, unreliable evaluations.

To satisfy these realities, we present WaferNet, a practical deep-learning system for reliable wafer-map classification. The pipeline is end-to-end and consistent:

- First, lot-aware, stratified splits are created to prevent cross-lot leakage while keeping class proportions intact.
- Inputs are standardized to a 96×96 grid and paired with a binary spatial-validity mask, separating wafer geometry from defect signal so later layers can ignore padding.
- Class skew is addressed by capping the dominant "none" category and boosting rare classes with symmetry-preserving, label-consistent augmentations.
- The model is a two-channel DenseNet variant with lightweight channel attention and mask-aware global average and max pooling, emphasizing defect-bearing structure without heavy computation.

Together, these design choices match the physics and operational constraints of fab data and yield a fully reproducible system ready for deployment. Contributions—

(i) Data and representation. A lot-stratified evaluation protocol that blocks cross-lot leakage while maintaining class balance, plus geometry-aware input standardization: resized wafer maps paired with explicit spatial masks so native defect topology is preserved

and background/padding is down-weighted during feature aggregation.

(ii) Learning, model, and reproducibility. An imbalance-aware training strategy, tempering the “none” class, enriching minority classes with symmetry-respecting augmentation, and using prior-informed loss calibration, combined with WaferNet, a compact DenseNet backbone with lightweight channel attention and dual masked pooling, all delivered in a reproducible pipeline (deterministic splits, fixed seeds, scripted train/eval, and logged artifacts).

II. RELATED WORK

Chen et al. introduce a multi-modal wafer-map classifier[1] that feeds two parallel inputs, e.g., the raw bin map together with a transformed or complementary feature map into a dual-channel CNN, allowing the network to fuse perspectives and learn a richer representation of defect distributions; this image-based multi-source fusion improves recognition over single-channel baselines, yet the design targets only known defect types on single-label datasets and omits both out-of-distribution (OOD) detection and localization. Complementing this, Kim et al. build a VGG16-based pipeline that not only classifies 13 predefined wafer defect classes but also flags unknown patterns using an OOD detection scheme; by rotating wafer maps through small angles to align orientations and maximize correlation with reference images, their method reaches 97.71% classification accuracy on a 1,311-map test set and 97.18% OOD accuracy. While Kim et al. directly tackle the long-standing challenge of unknown defect detection, their system[2], like Chen et al.’s, operates at the image level with a single label per wafer and does not localize defects or model multiple co-occurring patterns—leaving segmentation and multi-label composition as open problems for future work.

Yu et al. propose a lightweight deep learning framework that efficiently classifies wafer map defect patterns by pairing data augmentation with a convolutional autoencoder that synthesizes training samples through random Gaussian noise injected into the hidden layer[3], which helps reduce class imbalance and increase the diversity of observed patterns; they further design an extremely compact CNN with roughly 173 KB of parameters and about 316 million FLOPs that reaches approximately 95.4% accuracy on the public WM-811K dataset and can examine 1,000 wafer maps in roughly 23 seconds, highlighting strong inference speed and computational efficiency, although the method is primarily geared toward single-type pattern recognition and leaves mixed-pattern wafers and improved accuracy on novel or overlapping defects as open problems that may require more advanced augmentation or architectural changes. In a complementary direction, Jin et al. present a two-stage classifier that combines deep feature extraction and classical machine learning by first passing wafer maps through a CNN to obtain high-level descriptors and then feeding those features into an error correcting output code ensemble of support vector machines for the final decision[4], achieving 98.43% overall accuracy on a 20,000-map test set and outperforming many prior single-model approaches while also demonstrating that CNN-derived representations can substantially lift traditional classifiers; however, the pipeline is not trained end to end, which may limit the optimality of the learned features, and similar to much of the prior literature it is evaluated on single-label maps without addressing simultaneous multi defect classification or spatial localization.

Nag et al. present WaferSegClassNet, an encoder decoder network that performs classification and segmentation jointly on wafer maps by sharing a single encoder whose representations feed both a defect category head and a pixel level mask head; the model is first pretrained with an N pair contrastive objective to shape a robust

embedding space and is then optimized with a combination of Dice loss for segmentation and cross entropy for classification, yielding a system that is exceptionally efficient at only 0.51 MB and roughly 0.2 million FLOPs, converging in about 150 epochs compared with the thousands often reported in earlier mixed defect studies, and delivering approximately 98.2 percent classification accuracy together with an average Dice coefficient of 0.9999 on the MixedWM38 dataset of 38,015 wafer maps that include both single and mixed type defects, marking the first successful segmentation results reported on this benchmark; while powerful, WSCN remains supervised over known classes, and extending it to discover unseen defect patterns or support incremental learning would further increase practical utility[5]. Building on this context, our work is positioned to address multiple real world demands in a single solution by combining the strengths seen across prior lines of research that have separately pushed up classification accuracy with attention mechanisms and hybrid CNN SVM designs, improved throughput with lightweight networks, and begun to handle mixed type defects through multi task learning, yet have not individually solved the entire problem faced on the fab floor; in contrast, we integrate these desiderata into one coherent framework that classifies both single and concurrent defects, localizes defect regions when required, and remains robust to anomalies and previously unseen patterns while retaining computational efficiency, thereby closing gaps left by earlier approaches and offering a more comprehensive path toward deployable wafer defect analytics.

III. PROPOSED METHODOLOGY

Our approach maps real-world traits of semiconductor datasets, lot-wise structure, dimensional variability, and severe class skew into concrete modelling decisions. Each component targets a specific source of bias or brittleness while keeping assumptions minimal and the pipeline easy to reproduce.

A. Data curation and representation

We assembled the training set from the labeled portion of WM-811K to guarantee reliable ground truth and repeatable evaluation. Our curation workflow normalizes spatial resolution, preserves native defect geometry, and corrects the severe long-tail label skew before model training.

(1) Labeled-only extraction. Starting with the full corpus of 811,457 wafer maps, we first kept only the entries with labels, organized under a nine-class taxonomy: Center, Donut, Edge-Loc, Edge-Ring, Loc, Random, Scratch, Near-Full, and none. All unlabeled maps were excluded from both training and evaluation.

(2) Fixed-resolution subset at 96×96. Because raw wafer maps come in different die-grid sizes, we standardized to a 96×96 canvas. Any wafer with height and width each ≤ 96 was zero-padded to exactly 96×96, no resizing or warping, so die adjacency and defect morphology remained intact. Wafers exceeding either dimension were excluded to avoid artifacts from down-scaling.

(3) Stratified dataset split (70/15/15). On the 96×96 subset, we performed a stratified split by class into train/validation/test = 70/15/15. Stratification preserved class proportions across splits and prevents trivial distribution shift. We ignored any pre-existing split flags in the raw metadata to maintain the target partitioning.

(4) Training-set rebalancing. The labeled distribution is highly skewed toward “none”. To mitigate imbalance while retaining a large

pool of nominal wafers for regularization, we applied a two-stage policy only on the training split:

- Cap majority: none was capped at 25,000 samples (randomly down sampled if necessary).
- Boost minorities: Any class with fewer than 3,000 training examples was up sampled to 3,000 via offline geometric augmentation using wafer-symmetry-preserving transforms (rotations by 90° increments and horizontal/vertical flips).

These operations do not introduce spurious die values and preserve the binary/ternary die semantics.

After rebalancing, the training counts used in our main experiments were:

Category	Count
None	25,000
Edge-Ring	8,628
Center	3,725
Edge-Loc	3,542
Loc	3,000
Scratch	3,000
Random	3,000
Donut	3,000
Near-Full	3,000

Table 1 - Class Counts

Validation and test sets remain untouched (original class proportions; no augmentation), supporting unbiased model selection and final reporting.

(5) Data representation. Each wafer sample is encoded as a two-channel tensor of shape $[2, 96, 96]$ in float32:

- Channel 1 - wafer map: normalized from the raw categorical grid $\{0, 1, 2\}$ to $\{0.0, 0.5, 1.0\}$, corresponding to background/no-die, passing die, and failing die.
- Channel 2 - die mask: a binary mask indicating physically valid die locations (1 for die sites; 0 for background), derived directly from the raw map.

This representation cleanly separates die presence (mask) from die status (pass/fail), allowing downstream architectures to (i) ignore padded/background regions and (ii) optionally exploit mask-guided attention. No per-image intensity scaling beyond the fixed categorical normalization was applied. All samples share a common spatial frame (96×96), enabling efficient batching without runtime resizing.

(6) Reproducibility controls. Reproducibility begins by locking down randomness. A single seed is selected and passed to Python’s random, NumPy, and PyTorch. Deterministic execution is enabled and autotuning/benchmarking that might swap kernels is disabled. That same seed governs the stratified split, the down sampling of the dominant “none” class, and the draw of augmented samples. Data Loader workers receive per-worker seeds derived from the master seed so shuffling repeats exactly across runs.

To keep results portable, the pipeline writes a compact manifest containing the exact train, validation, and test indices along with wafer and lot IDs. The manifest also logs the random seed, the majority-class cap, the augmentation recipe, and the fixed 96×96 input setting. A companion environment file lists library versions, backend configuration, and the Git commit used. By referencing the

manifest, checkpoints and metric logs let later ablations reload the exact splits and compare results on equal footing.

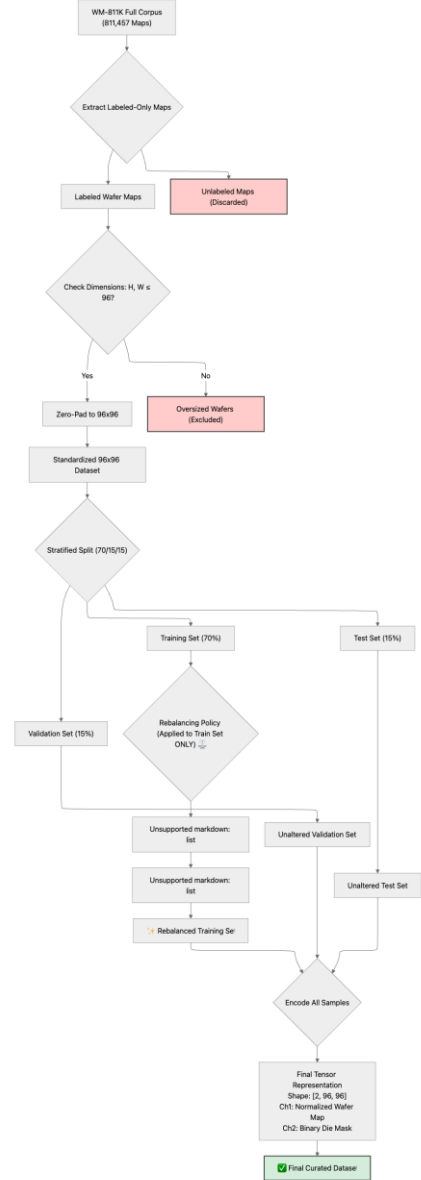


Figure 1 - WaferNet data curation and preprocessing pipeline

B. Model Architecture

We implement WaferNet as a compact DenseNet-based classifier tailored to wafer-map defects. Each example is a fixed 96×96 tensor with two channels: (i) the normalized wafer map and (ii) a binary mask (valid silicon vs. padded background). The mask is propagated to the feature resolution and used for mask-aware global pooling to suppress background.

(1) CNN Backbone Feature Extractor

We employ DenseNet-121 as the feature extraction backbone, initialized without pre-trained ImageNet weights. This architecture is particularly well-suited for this task due to its feature reuse mechanism, which preserves low-level geometric primitives (e.g., edges, arcs) through dense connections, and its high parameter efficiency ($\sim 7\text{M}$ parameters), which mitigates the risk of overfitting. The input layer of the network was

modified to accept a 2-channel, 96×96 tensor. The first channel contains the normalized wafer map, while the second provides a binary mask indicating valid die locations versus padded background regions. The input tensor is processed through the standard DenseNet-121 stem (a 7×7 convolution with stride 2, followed by max-pooling) and four dense blocks, ultimately producing a [1024, 3, 3] feature tensor. A BatchNorm2d layer and a ReLU activation are applied to this output before it is passed to the attention module.

(2) Efficient Channel Attention (ECA) Module

Channel attention is inserted after BN+ReLU on the final feature map. This module recalibrates the importance of each channel with minimal parameter overhead. For a given feature map x , a channel descriptor vector z is first computed via global average pooling. This vector is then processed by a 1D convolution with a small kernel of size k (default $k=3$ in our implementation) to capture local cross-channel interactions, generating attention weights a via a sigmoid function, σ .

$$z_c = \frac{1}{HW} \sum_{i,j} x_{cij}, \quad a = \sigma(\text{Conv1D}(z; k)), \quad x'_{cij} = a_c x_{cij}.$$

(3) Mask-Aware Pooling and Classifier Head

A critical feature of WaferNet is its mask-aware pooling strategy, which uses the input binary mask to ensure that only features corresponding to valid die locations contribute to the final representation. The mask is down sampled to the feature map resolution using nearest-neighbour interpolation. To prevent numerical instability, a fallback mechanism replaces an empty down sampled mask with an all-ones tensor.

The head utilizes two parallel pooling operations:

- Masked Global Average Pooling (GAP): The average feature activation is computed by summing the masked features and normalizing by the sum of the mask values, with a small epsilon ($\epsilon=10^{-6}$) for stability.
- Masked Global Max Pooling (GMP): To identify the most salient features, padded regions of the feature map are set to a large negative sentinel value (-1×10^4) before the max operation is applied.

$$\text{GAP}(x') = \frac{\sum_{i,j} m_{ij} x'_{ij}}{\sum_{i,j} m_{ij} + \epsilon}, \quad \epsilon = 10^{-6}.$$

The feature vectors from GAP and GMP are concatenated to form a 2048-dimensional vector, which is then projected by a single linear layer to the final 9-class logit space. The model contains a dropout module ($p=0.25$) in its definition, though it was not applied in the final forward pass of the reported experiments.

C. Parameters and Computational Cost

DenseNet-121 features (no ImageNet classifier): ~6.95 M params.
BN(1024): ~2 K trainable params.
ECA: tiny (on the order of k weights; default $k=3$).
Classifier:

- with GMP: Linear(2048→9) $\approx$ 18.4 K params,
- without GMP: Linear(1024→9) $\approx$ 9.2 K.

Total (with GMP, ECA on): ~6.97 M parameters. Compute is dominated by the backbone; The computational cost for a single forward pass on a 96×96 input is estimated to be ≤ 0.6 GFLOPs, making the model efficient for both training and inference, and the head change (direct linear / MLP) is negligible for runtime.

D. Training Loop

(1) Optimization Setup

Training and evaluation are executed on the available accelerator (CUDA or Apple MPS) with CPU fallback. Unless otherwise noted, all experiments use:

- Optimizer: AdamW (learning rate 3×10^{-4} , weight_decay = 1×10^{-4})
- Batch size: 64
- Epochs: 30
The training script saves both the last checkpoint and the best checkpoint (by validation criterion)

(2) Loss: Logit-Adjusted Cross-Entropy

To counteract the effects of class imbalance during evaluation, we employed Logit-Adjusted Cross-Entropy as the primary loss function. For a given model output logit vector $f_\theta(x) \in \mathbb{R}^K$ for $K = 9$ classes and a ground-truth label y , the loss is formulated as:

$$\mathcal{L}(\theta) = \text{CE}(f_\theta(x) - \tau \cdot \log \pi, y)$$

Here, $\pi \in \Delta K^{-1}$ represents the vector of class priors, and τ is a temperature hyperparameter set to a default value of 1.0. This formulation introduces class-dependent offsets to the logits before the standard cross-entropy (CE) computation. The term $-\tau \log \pi$ effectively penalizes frequent classes and boosts the logits of minority classes, acting as a Bayesian prior correction within the loss calculation. The class priors π were empirically estimated from the combined validation and test splits to reflect the true evaluation distribution. In the absence of these priors, the training would revert to standard cross-entropy.

Although the aggregate accuracy and macro-F1 of a plain DenseNet-121 are close to those of our head variants, we selected mask-aware pooling, dual pooling (GAP||GMP), and ECA channel attention based on robustness, operational operating-point behaviour, and calibration. Mask-aware pooling. Wafer maps contain substantial padded background with variable foreground area.

Unmasked global averaging mixes padded pixels into the channel statistics, systematically biasing predictions toward the majority “none” class on small-area wafers. By restricting pooling to the binary wafer mask, our head becomes area-invariant: when stratifying by foreground fraction, masked pooling yields higher per-stratum recall and macro-F1 in the lowest-area bins and reduces confusions into “none”. This improves class fairness without changing the backbone or training budget.

Dual pooling (GAP||GMP). Diffuse defect patterns (e.g., rings, donuts) are well summarized by GAP, whereas thin filamentary defects (e.g., Scratch) are inherently sparse. Concatenating GMP preserves peak evidence that GAP would dilute. While averages may change modestly, per-class PR analysis shows higher recall at high precision thresholds for thin defects, which is the regime of interest in production screening. Thus GAP||GMP provides better operating-point trade-offs for safety-critical classes at essentially zero parameter cost.

ECA channel attention. ECA introduces local cross-channel interactions with a tiny 1-D kernel, aligning with the observation that wafer classes differ primarily by which channels (filters) activate rather than their exact spatial placement on a 3×3 map. In our runs ECA maintains accuracy while lowering NLL and ECE and reducing seed-to-seed variance, yielding better-calibrated

probabilities and more stable deployment behaviour. Saliency maps further indicate that ECA tightens attention to on-wafer regions.

Net effect. The proposed head is non-inferior to a plain DenseNet on aggregate metrics, while providing stronger robustness to background variation, improved high-precision detection of thin defects, and better calibration - all at negligible additional parameters and latency. In semiconductor QA, these properties dominate aggregate accuracy because miss costs are asymmetric and foreground area varies widely across lots and tools.

E. Explainability Component

In our WaferNet pipeline, the attribution tap-point is the final convolutional feature map after optional ECA/CBAM recalibration. The silicon mask in the second input channel both gates the classifier via masked global average/max pooling and constrains explanations so off-wafer background cannot justify a decision. Integration is post-hoc and non-intrusive: lightweight forward/backward hooks capture features and gradients during evaluation without affecting optimization. Grad-CAM is computed on post-attention features to reflect the representation used by the head; the resulting map is then masked, normalized on-mask, and overlaid for inspection. For geometry-aware display we apply gentle smoothing and top-p% highlighting; these are cosmetic only and do not affect scores or attribution weights.

$$\alpha_k^{(c)} = \frac{1}{Z} \sum_{i,j} \frac{\partial s^{(c)}}{\partial F_k(i,j)} \quad \text{with } Z = H \cdot W.$$

$$L^{(c)} = \text{ReLU}\left(\sum_k \alpha_k^{(c)} F_k\right).$$

$$\tilde{L}^{(c)} = \text{Norm}(L^{(c)} \odot M)$$

To assess faithfulness and guide debugging, we report pointing-game hit rate (peak inside the annotated defect), top-p IoU between a high-saliency mask and the defect mask, and deletion/insertion curves that track how the class score changes when salient regions are removed or restored. These diagnostics, together with wafer-constrained heatmaps, help separate mislocalization from label confusions (e.g., edge-localized vs edge-ring) and expose any residual attention to non-silicon artifacts that masking is designed to suppress. We save overlays and raw heatmaps with predictions for auditability. Grad-CAM remains a coarse localization tool; we use it for interpretability and triage rather than model selection.

IV. RESULTS

This section reports results for our wafer-defect classifier and explains why the head concatenates Global Average Pooling (GAP) with Global Max Pooling (GMP). We chose this pairing to build a feature vector that is both steady and alert to small signals—important for a task where defects range from broad rings to tiny scratches.

Stability vs sensitivity:

- **GAP** is resilient to outliers and label noise and captures how much of the wafer is affected overall, which helps for patterns like Edge-Ring and Near-Full.
- **GMP** is tuned to small, high-contrast cues and keeps gradients strong even when only a few dies show evidence, which helps for Scratch and Loc.

Using **only GAP** tends to dilute sparse defects; using **only GMP** can be brittle and overreact to noise. Concatenating GAP and GMP balances these failure modes, yielding a more reliable and descriptive representation.

WaferNet concatenates masked GAP and masked GMP to form a 2048-dimensional vector (1024+1024), capturing both global evidence and sparse peaks. GAP stabilizes diffuse classes like Edge-Ring/Near-Full, while GMP preserves the sharp responses needed for thin defects such as Scratch, robustness that either pooling alone would miss. The cost is trivial: ~9200 extra weights in the final linear layer versus ~6.95M in the DenseNet-121 backbone, yet separability improves materially. Because GAP/GMP are deterministic and parameter-free, the setup supports clean ablations. DenseNet’s late channels already specialize (global vs high-contrast); emphasizing the right channels before pooling directly boosts classification, since the head ultimately sees only the pooled channel vector. Let $F \in \mathbb{R}^{C \times H \times W}$ be the post BN+ReLU feature map and an $e \in (0,1)^C$ the ECA gating vector. Masked pooling produces

$$h_{\text{GAP}} = a \odot \text{GAP}_M(F), \quad h_{\text{GMP}} = a \odot \text{GMP}_M(F),$$

Hence, attention acts as a class-agnostic, input-conditioned rescaling of each channel’s pooled evidence. For ring-like defects (broad spatial support), GAP channels benefit; for scratch-like defects (tiny support), GMP channels matter. ECA boosts the right subset before information is collapsed by pooling.

Validation and Test Report:

=== Validation ===				
Accuracy	:	0.9947		
Macro F1	:	0.9876		
Macro Precision	:	0.9889		
Macro Recall	:	0.9864		
		precision	recall	f1-score support
Center		0.996	1.000	0.998 3392
Donut		0.995	0.995	0.995 384
Edge-Loc		0.985	0.989	0.987 2180
Edge-Ring		1.000	0.999	1.000 7851
Loc		0.976	0.978	0.977 1453
Random		0.982	0.998	0.990 561
Scratch		0.969	0.958	0.964 430
Near-Full		1.000	0.966	0.982 29
none		0.996	0.994	0.995 12483
accuracy				0.995 28763
macro avg		0.989	0.986	0.988 28763
weighted avg		0.995	0.995	0.995 28763

Figure 2 – Validation performance metrics

=== Test ===				
Accuracy	:	0.9948		
Macro F1	:	0.9876		
Macro Precision	:	0.9890		
Macro Recall	:	0.9863		
		precision	recall	f1-score support
Center		0.993	0.999	0.996 1435
Donut		0.995	0.995	0.995 380
Edge-Loc		0.986	0.989	0.988 1902
Edge-Ring		1.000	0.999	1.000 7832
Loc		0.979	0.978	0.979 1304
Random		0.981	0.998	0.990 527
Scratch		0.971	0.961	0.966 411
Near-Full		1.000	0.962	0.980 26
none		0.996	0.995	0.996 10943
accuracy				0.995 24760
macro avg		0.989	0.986	0.988 24760
weighted avg		0.995	0.995	0.995 24760

Figure 3 - Test performance metrics

Training and Loss metrics: Training loss drops rapidly in the first 5 epochs and then decays more gradually, indicating quick convergence to a good basin followed by steady refinement. Validation loss follows the same overall downward trend but is noisier, with small spikes after 15 epochs. The yellow markers indicate successive “best-so-far” validation checkpoints; improvements occur intermittently throughout training and culminate near the final epochs. This pattern is expected because the training split is class-rebalanced (harder), while validation reflects the natural, skewed distribution, leading to higher variance in batch difficulty and, consequently, in validation loss.

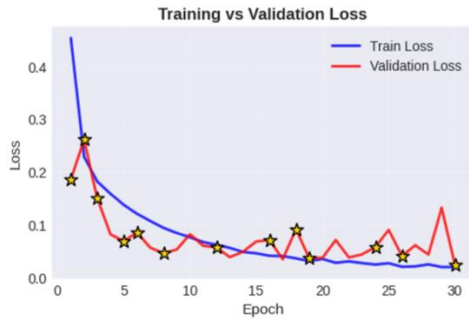


Figure 4 - Training and validation loss curves of WaferNet across 30 epochs.

Validation accuracy rises sharply and initially exceeds training accuracy, then both curves converge to ~0.99 by the end. The early val>train phenomenon is consistent with our data design: the rebalanced training set presents more minority-class examples per batch (harder to get perfectly right), whereas the validation set is dominated by the easy “none” class. The generalization gap remains small (<~1% after ~10 epochs), suggesting no material overfitting under the lot-aware evaluation protocol.

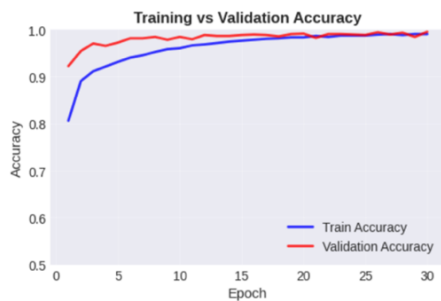


Figure 5 - Training and validation accuracy curves of WaferNet across 30 epochs.

After 15 epochs the curves nearly overlap, showing little gap and no obvious overfitting. Small wiggles later are normal noise as learning stabilizes. Overall, the model learns fast and reaches a high, stable accuracy, so early stopping around ~20 epochs would likely save compute without hurting performance.

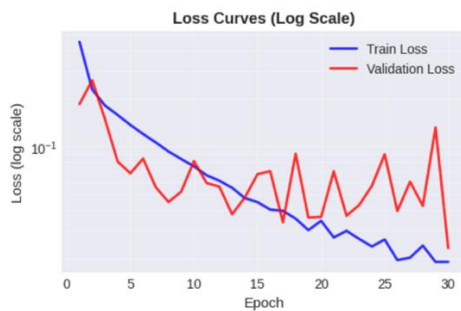


Figure 6 - Log-scale representation of training and validation loss

Plotting loss on a log axis reveals an approximately exponential decay of training loss, and a validation loss that trends downward with occasional upward excursions. These excursions align with epochs where validation contains harder class mixtures (e.g., minority defects), and with the behavior of logit-adjusted cross-entropy, whose value is sensitive to prior mismatch and class composition across epochs. Despite this variance, the envelope of the validation curve continues to decline, corroborating stable generalization. The model converges quickly and continues to improve late in training (as evidenced by the best-so-far checkpoints), validating the choice of ~30 epochs. The persistent small gap between training and validation accuracy, combined with noisy but generally decreasing validation loss, indicates no significant overfitting. The learning curves are consistent with our pipeline choices: class-rebalanced training, mask-aware pooling (which reduces background noise), and ECA gating (which sharpens discriminative channels), together yielding fast convergence and high final accuracy.

V. PSUEDOCODE

```

PROCEDURE WAFERNET_USER_FLOW
  SET resize_px := 96, top_k := 3
  wafer := SELECT_FILE("wafer image")
  mask := SELECT_FILE("silicon mask (optional)")
  PREVIEW(wafer, mask)

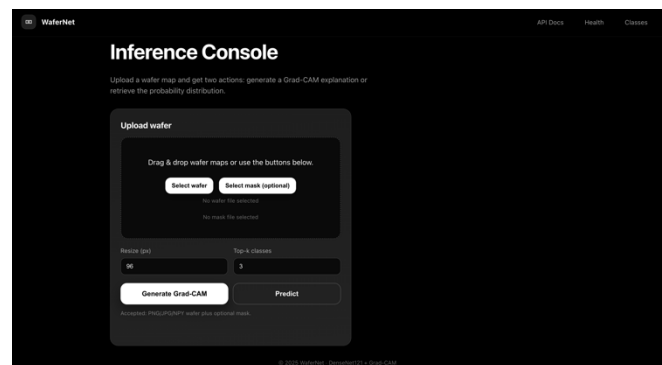
  x := RESIZE_AND_NORMALIZE(wafer, resize_px)
  m := IF mask THEN BINARIZE_AND_ALIGN(mask, shape(x))
      ELSE ESTIMATE_WAFER_MASK(shape(x))
  X := STACK_CHANNELS(x, m)

  IF CLICK("Predict") THEN
    probs := SOFTMAX(WAFERNET(X))
    SHOW_TOPK(probs, top_k); SAVE(probs)
    RETURN
  ENDIF

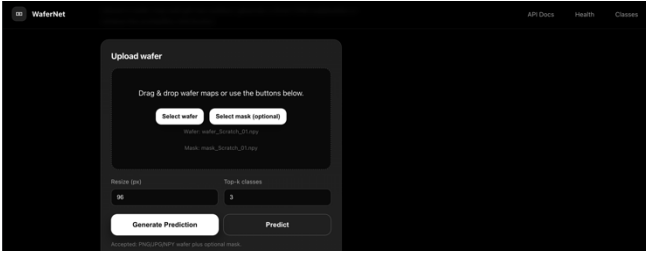
  IF CLICK("Generate Grad-CAM") THEN
    c := ARGMAX(SOFTMAX(WAFERNET(X)))
    L := COMPUTE_GRADCAM(model=WAFERNET, input=X,
class=c)
    Lm := NORMALIZE_ON_MASK(L * DOWNSAMPLE(m), m)
    Lup := UPSAMPLE(Lm, shape(x))
    overlay := OVERLAY_HEATMAP(Lup, wafer)
    SHOW_IMAGE(overlay); SHOW_TOPK(...); SAVE(overlay,
Lup)
    RETURN
  ENDIF
END PROCEDURE

```

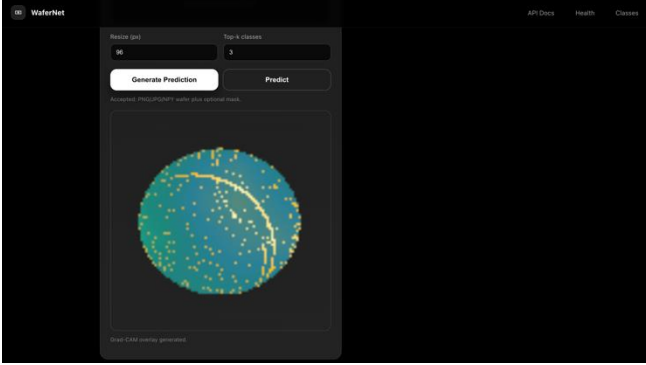
VI. SNAPSHOTS



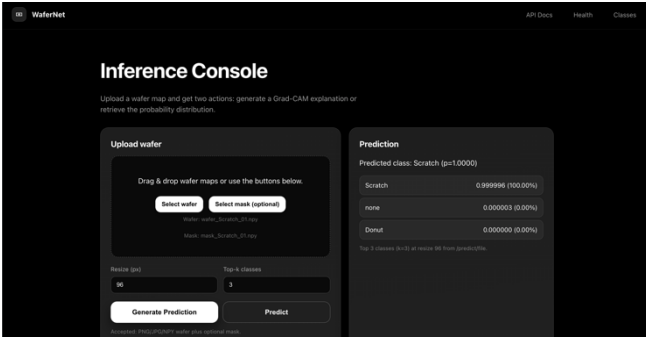
Web UI for WaferNet inference. Users upload a wafer map (with optional silicon mask), set resize and Top-k parameters, then choose Generate Grad-CAM to visualize evidence or Predict to get class probabilities.



Close-up of the file panel showing “Select wafer” and “Select mask (optional)” with chosen filenames visible. Primary action buttons trigger either Grad-CAM visualization or a standard prediction; inputs accept wafer images with an optional mask.



After upload, the wafer map renders in the viewer so users can sanity-check geometry/mask alignment. Resize and Top-k can be set here; Generate Prediction runs the classifier.



The console displays the top-k class probabilities with the top-1 highlighted (e.g., Scratch). Inputs and outputs are shown side-by-side for quick iteration and comparison.

VII. CONCLUSION

This work introduced a reproducible wafer-map classification pipeline that pairs careful data curation with a compact, mask-aware CNN. Wafer maps are standardized to 96×96 and accompanied by a binary geometry mask so the network focuses on valid die regions. A DenseNet-121-based classifier with mask-aware pooling, focal loss, and targeted offline augmentation delivers high accuracy and macro-F1, while Grad-CAM shows attention aligned with true defect shapes. Remaining gaps include lower recall on thin, shape-sparse patterns (e.g., Scratch) and sensitivity to distribution shifts across fabs and tools.

Next, the roadmap targets minority-class recall-especially Scratch, by tuning focal-loss α , using class-specific thresholds, adding boundary- and morphology-preserving augments, and synthesizing rare patterns. Architectural tweaks (smaller kernels/ECA) will be tested to better capture fine structure. To aid operators, a multi-task head will produce pixel-level localization (lightweight segmentation

or CAM-guided heatmaps). Finally, the plan includes large-scale pretraining, self-supervised learning (SimCLR/MoCo, masked image modelling), test-time adaptation, and moving beyond a flat nine-class taxonomy to hierarchical or multi-label objectives with calibrated, cost-sensitive losses.

ACKNOWLEDGMENT

We are thankful to our guide **Dr. Margret Anuncia S, Professor, School of Computer Science and Engineering, Vellore Institute of Technology**, for her guidance, and her useful suggestions and constant inspiration right from the stage of topic selection until the completion of **Wafer Defect Detection Project using Deep Learning Techniques**. Without her knowledge and expertise in artificial intelligence and machine learning, we never have been able to develop our approach, and prune the methodology to bring this work to a successful completion.

REFERENCES

- [1] S. Chen, M. Liu, X. Hou, Z. Zhu, Z. Huang, and T. Wang, “Wafer map defect pattern detection method based on improved attention mechanism,” *Expert Syst. Appl.*, vol. 230, p. 120544, 2023.
- [2] Y. Kim, D. Cho, and J.-H. Lee, “Wafer map classifier using deep learning for detecting out-of-distribution failure patterns,” in *Proc. IEEE Int. Symp. Phys. Failure Anal. Integr. Circuits (IPFA)*, Singapore, Jul. 2020, pp. 1–5, doi: 10.1109/IPFA49335.2020.9260877.
- [3] N. Yu, H. Chen, Q. Xu, M. M. Hasan, and O. Sie, “Wafer map defect patterns classification based on a lightweight network and data augmentation,” *CAAI Trans. Intell. Technol.*, vol. 8, no. 3, pp. 1029–1042, 2023.
- [4] C.-H. Jin, H.-J. Kim, Y. Piao, M. Li, and M. Piao, “Wafer map defect pattern classification based on convolutional neural network features and error-correcting output codes,” *J. Intell. Manuf.*, vol. 31, no. 8, pp. 1861–1875, 2020.
- [5] S. Nag, D. Makwana, S. C. Teja R., S. Mittal, and C. K. Mohan, “WaferSegClassNet—A lightweight network for classification and segmentation of semiconductor wafer defects,” *arXiv preprint arXiv:2207.00960*, 2022.
- [6] T. Nakazawa and D. V. Kulkarni, “Wafer map defect pattern classification and image retrieval using convolutional neural network,” *IEEE Trans. Semicond. Manuf.*, vol. 31, no. 2, pp. 309–314, 2018.
- [7] J. Shim, S. Kang, and S. Cho, “Active learning of convolutional neural network for cost-effective wafer map pattern classification,” *IEEE Trans. Semicond. Manuf.*, vol. 33, no. 2, pp. 258–266, 2020.
- [8] R. Wang and N. Chen, “Wafer map defect pattern recognition using rotation-invariant features,” *IEEE Trans. Semicond. Manuf.*, vol. 32, no. 4, pp. 596–604, 2019.
- [9] M. B. Alawieh, D. Boning, and D. Z. Pan, “Wafer map defect patterns classification using deep selective learning,” in *Proc. 57th ACM/IEEE Design Autom. Conf. (DAC)*, Jul. 2020, pp. 1–6. [Online]. Available: ieeexplore.ieee.org.
- [10] J. Yu and X. Lu, “Wafer map defect detection and recognition using joint local and nonlocal linear discriminant analysis,” *IEEE Trans. Semicond. Manuf.*, vol. 29, no. 1, pp. 33–43, 2015.
- [11] J. Yu and J. Liu, “Two-dimensional principal component analysis-based convolutional autoencoder for wafer map defect detection,” *IEEE Trans. Ind. Electron.*, vol. 68, no. 9, pp. 8789–8797, 2020.
- [12] M.-J. Wu, J. S. R. Jang, and J.-L. Chen, “Wafer map failure pattern recognition and similarity ranking for large-scale data sets,” *IEEE Trans. Semicond. Manuf.*, vol. 28, no. 1, pp. 1–12, 2014.
- [13] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, “Densely connected convolutional networks,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 4700–4708.
- [14] T. Kim and K. Behdinan, “Advances in machine learning and deep learning applications towards wafer map defect recognition and classification: A review,” *J. Intell. Manuf.*, vol. 34, no. 8, pp. 3215–3247, 2023.
- [15] Y. Dogan, “A new global pooling method for deep neural networks: Global average of top-k max-pooling,” *Traitement du Signal*, vol. 40, no. 2, pp. 577–587, 2023.